Async-патерни + Streams + backpressure

Promises · microtasks · AsyncLocalStorage · AbortController · Streams · pipeline

Заняття №2

Node.js Pro

Лектор
Кирило Орлов

ПЛАН ЗАНЯТТЯ



- Асинхронність у часі: Promises, microtasks vs macrotasks, async/await
- Контекст запиту: AsyncLocalStorage; скасування: AbortController + structured concurrency
- Streams у пам ′яті: Readable / Writable / Transform / Duplex, highWaterMark
- Backpressure і pipeline — демо: процес падає від нестачі пам ′яті vs ті самі ~10 GB крізь ~100 MB RAM; наприкінці — коли Web Streams, а коли node:stream

СТЕК НА СЬОГОДНІ

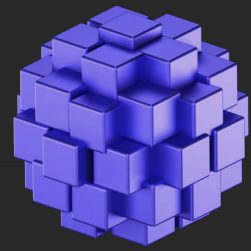
Група 1 — про ЧАС (коли що виконається): Promises → microtasks → AsyncLocalStorage → AbortController

Група 2 — про ПАМ′ ЯТЬ (скільки даних тримаємо): Streams (Readable / Writable / Transform / Duplex) → pipeline

Спільна ідея обох груп — зворотний тиск (backpressure)

В async це AbortSignal, у streams — write() === false

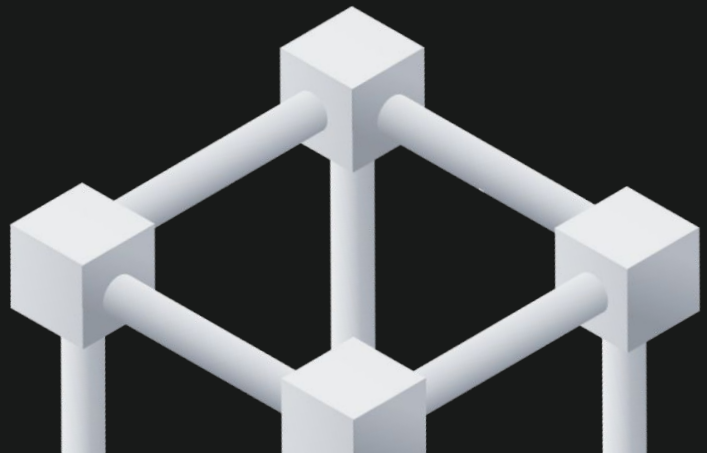
Мета заняття: протягнути ~10 GB даних крізь ~100 MB RAM



Разом 6 технологій: чотири про час виконання, дві про пам′ ять під навантаженням

АСИНХРОННІСТЬ: КЕРУЄМО ЧАСОМ

PROMISES → MICROTASKS →
ASYNCLOCALSTORAGE → ABORTCONTROLLER



РЕКУРСИВНИЙ NEXTTICK = ПРИЧИНА №1 EVENT-LOOP LAG

Event loop крутить 6 фаз по колу: timers → pending → idle/prepare (службова) → poll (I/O) → check → close

Між кожними двома фазами спорожнюються ПОВНІСТЮ дві черги: спершу вся nextTick, потім уся microtasks (Promise-колбеки)

Рекурсивний nextTick: колбек планує сам себе → черга ніколи не порожня → loop застряг перед наступною фазою

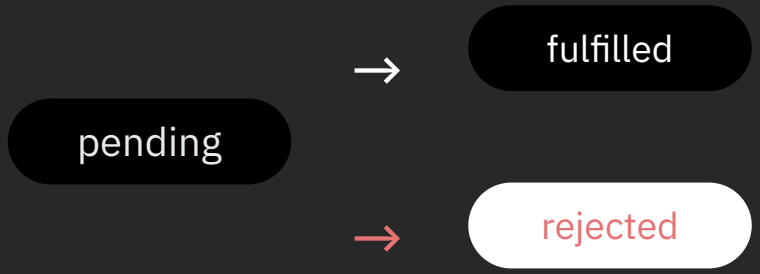
Наслідок: I/O, таймери, нові з ′єднання не отримують керування = starvation (голодування фаз)

Важку роботу ріж на порції через setImmediate (виконається у фазі check, дасть loop дихати), НЕ через nextTick

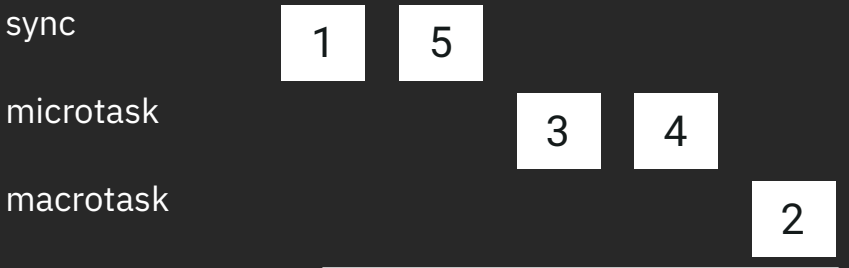
PROMISES: СТЕЙТ-МАШИНА + ПОРЯДОК-КВІЗ

```
console.log(1);                                  // sync
setTimeout(() => console.log(2), 0);             // macrotask (черга таймерів)
Promise.resolve().then(() => console.log(3));    // microtask
queueMicrotask(() => console.log(4));            // microtask
console.log(5);                                  // sync
```

СТЕЙТ-МАШИНА · pending → fulfilled АБО rejected · перехід одноразовий, незворотний



ПОРЯДОК ВИКОНАННЯ



setTimeout(..., 0) ≠ негайно: чекає, поки спорожніє microtask-черга

→ 1, 5, 3, 4, 2

«ЧЕРГИ БАНКУ»: MICROTASK VS MACROTASK

Один клієнт біля вікна = одна задача: її тіло виконується синхронно ДО КІНЦЯ і не переривається (перша задача — стартовий скрипт, далі — колбеки)

Дрібна доробка перед викликом наступного клієнта = microtask (.then / queueMicrotask)

Термінове розпорядження керівника = process.nextTick — робиться ПОПЕРЕД усіх звичайних доробок

Оголошення «наступний!» у гучномовець = macrotask (setTimeout / I/O — новий «клієнт»)

Правило: операціоніст закриває ВСІ накопичені доробки (усю microtask-чергу) і лише тоді гукає наступного (одна macrotask)

Доробка породжує нову доробку → черга не порожніє → вікно стоїть, клієнти не рухаються = starvation

ТРИ SENIOR-ПАСТКИ PROMISES

Три пастки, які ловлять навіть досвідчених

Пастка 1 — Thenable

- Promise.resolve(об'єкт-з-.then)
- розгортає за ДВА microtask-тіки замість одного
- зайва затримка і збитий порядок

Пастка 2 — unhandledRejection

- reject без .catch → краш процесу
- Node ≥15: це краш, а не warning
- reject не можна «лишити на потім»

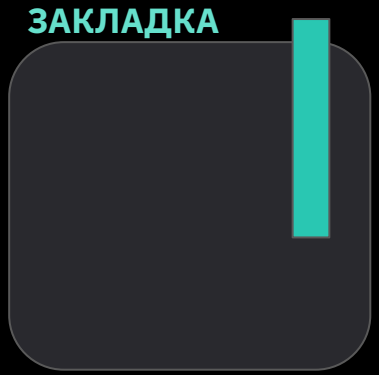
Пастка 3 — .finally()

- хук прибирання ресурсів (cleanup)
- НЕ бачить значення успіху
- НЕ ковтає помилку — вона летить далі

Та сама microtask-модель, що в браузері (специфікація HTML). Виняток — process.nextTick: у браузері нативно його немає, старі бандлери підставляли поліфіл через setTimeout (це вже macrotask)

ASYNC/AWAIT = MICROTASK CHECKPOINT

async-функція завжди повертає Promise; кожен await розрізає її на окремі .then-колбеки



Метафора: await = закладка в книжці — функція лишає мітку, де спинилась, і повертається сюди, коли Promise готовий виконати решту.

Checkpoint 1
до await ① — синхронний старт

.then-колбек #1
решта коду після await ① — окремий колбек

.then-колбек #2
reject виринає як throw, ловиться в catch

◄ await ①
розріз 1 → перехід у microtask

◄ await ②
розріз 2 → перехід у microtask

ASYNC/AWAIT: ДВІ ПАСТКИ + КОМБІНАТОРИ

Пастка 1 — await у циклі: N запитів ідуть послідовно, кожен чекає попереднього → p99 гірший у 10-40×

Фікс: Promise.all (усі одразу) або bounded-concurrency (пачками по k паралельно)

Пастка 2 — race-condition: паралельні записи у спільний ресурс без транзакції → загублене оновлення (lost update)

all — fail-fast: перший reject одразу кидає, решту не чекає

allSettled — чекає ВСІХ, повертає масив {status, value|reason}, сам нічого не кидає

race — перший, що завершився будь-як (успіх чи reject)

any — перший УСПІШНИЙ (усі reject → AggregateError)

ASYNCLOCALSTORAGE: БРАСЛЕТ ДЛЯ ЗАПИТУ

AsyncLocalStorage (node:async_hooks) — сховище, прив ′язане до «логічного» запиту, а не до глобальної змінної

Проблема без нього: requestId / userId доводиться прокидати параметром крізь усі 5 шарів (prop drilling)

als.run(store, fn) — виконує fn, і весь ланцюг всередині бачить store (звичайний об ′єкт, напр. { requestId, userId })

als.getStore()?.requestId — дістає той самий store будь-де глибше, без передачі параметром

Контекст сам зберігається крізь async-межі — await, .then, setTimeout, I/O: значення не губиться

Анти-патерн: модульна змінна let currentRequestId під конкурентним навантаженням плутає запити A і B

ALS: МЕЖІ + ЦІНА (ASYNCCONTEXTFRAME)

getStore() поза run() → undefined; завжди звертайся через ?. + fallback

enterWith(store) ставить контекст «на місці», без обгортки run() — тече у сусідні операції; уникай у бібліотеці

snapshot() «заморожує» поточний контекст, щоб віддати його пізнім колбекам (event-emitter, пул), які інакше його втратять

Межа: контекст НЕ перетинає worker_thread чи інший процес — туди передавай значення явно

async_hooks — низькорівневий API, на якому побудований ALS; напряму його не пишемо

Ціна/версія: Node 24 — AsyncContextFrame (швидка реалізація) дефолт; Node ≤22 — лише за прапорцем --experimental-async-context-frame

ABORTCONTROLLER: ПУЛЬТ «СТОП»

AbortController = пульт: .signal роздаєш споживачам (fetch, таймер, стрім), .abort(reason) — тиснеш «стоп» одразу для всіх

reason — довільна причина скасування (напр. new Error('timeout')), яку отримає кожен споживач

AbortSignal — стандартний веб-об ′єкт (той самий, що в браузері); fetch і більшість async-API вміють його слухати

Готові фабрики сигналів: AbortSignal.timeout(ms) сам спрацює через ms; AbortSignal.any([...]) — від будь-якого зі списку; signal.throwIfAborted() кине, якщо вже скасовано

STRUCTURED CONCURRENCY: ABORTCONTROLLER + PROMISE.ALL

Structured concurrency = принцип «діти не переживають батька»: батько впав → усі паралельні під-задачі скасовуються

У JS цього нема нативно — збираємо руками з AbortController + Promise.all

Спільний signal на всі гілки: перша помилка → ac.abort() → скасувати сиблінгів одразу (fail-fast, без чекання решти)

Без спільного signal сиблінги лишаються orphan (осиротілі fetch'і): тримають сокети, а пізній reject → unhandledRejection → краш (Node ≥15)

Та сама ідея, що signal у pipeline(..., { signal }) для стрімів; врахуй баг nodejs/node#57736 (any+timeout — тримай запасний явний таймер)

STREAMS: КЕРУЄМО ПАМ'ЯТТЮ

READABLE / WRITABLE / TRANSFORM / DUPLEX → BACKPRESSURE → PIPELINE



STREAMS: 4 ТИПИ = ШЛАНГ ЗАМІСТЬ ВІДЕР

Стрім = шланг замість відер: дані течуть порціями (chunks), у RAM одночасно лише обсяг «шланга», а не весь файл

Readable — джерело (файл, HTTP body, курсор БД); два режими: paused (тягнеш вручну .read()) та flowing (само штовхає через 'data' / for await)

Writable — приймач; write(chunk) повертає boolean: false = буфер повний, пригальмуй

Transform — стрім-функція: вихід = перетворення входу (gzip / парсинг NDJSON / шифр)

Duplex — читання і запис в одному об ′єкті, дві сторони незалежні (TCP-сокет)

Помилку в _transform віддавай через callback cb(err), а не throw — інакше вона стає uncaughtException і роняє процес

HIGHWATERMARK, OBJECTMODE, FOR AWAIT

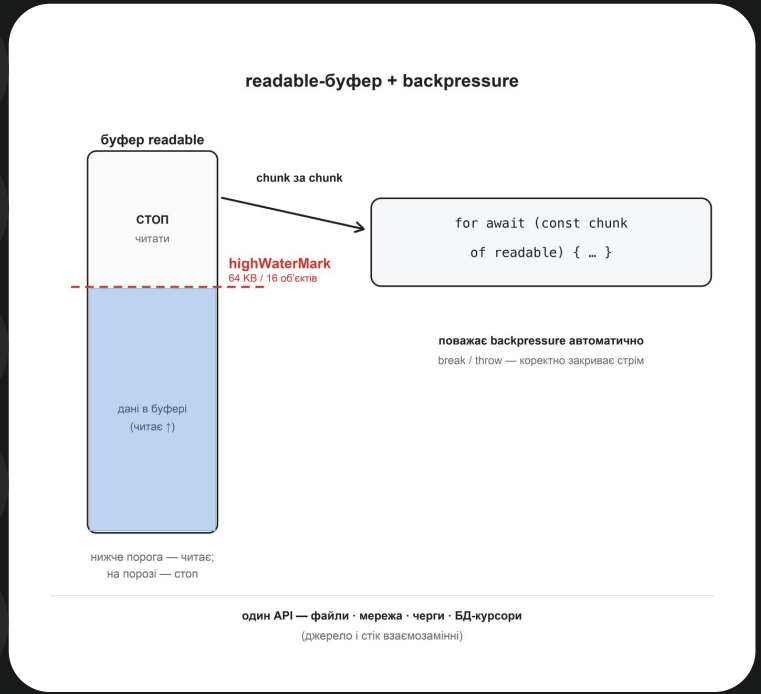
highWaterMark — поріг буфера, за яким стрім каже «стоп»: дефолт 64 KB у байтовому режимі АБО 16 елементів у objectMode

Це НЕ ліміт пам ′яті, а точка перемикання «стоп читати / стоп писати» — сигнал backpressure (зворотного тиску)

objectMode — чанки як JS-об ′єкти (не Buffer); тоді highWaterMark рахує їхню КІЛЬКІСТЬ, а не байти

for await (const chunk of readable) ітерує стрім, автоматично поважаючи backpressure; break / throw коректно закриває стрім (ручний цикл на 'data' цього не гарантує → leak)

Один і той самий стрім-API — файли, мережа, черги, БД-курсори: джерело і стік взаємозамінні



BACKPRESSURE: КАСА СУПЕРМАРКЕТУ

Readable = касир (виробляє дані)

Writable = пакувальник (пакує у своєму темпі)

chunk = товар; highWaterMark = місткість стрічки між касиром і пакувальником

write() === false = стрічка повна → касир зупиняється («стоп»)

подія 'drain' = пакувальник розгріб чергу → «давай далі»

Ігнорувати false і писати далі = товари сиплються на підлогу = переповнення й краш RAM (OOM, out of memory)

pipeline(src, dst, { signal }) = менеджер, що синхронізує темп і за signal скасовує всю касу; ручний .pipe() без цього громадиться до падіння

Крайній випадок: пакувальник спинився зовсім

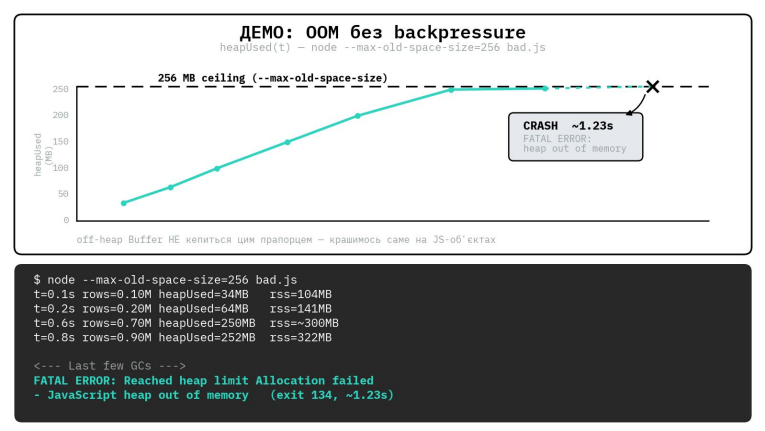
- **pipeline({ signal }) скасовує всю касу — коректно, без витоку**
- **ручний .pipe() громадиться, поки не впаде від нестачі пам′яті**

/LIVE CODING:



ДЕМО: OOM БЕЗ BACKPRESSURE vs PIPELINE

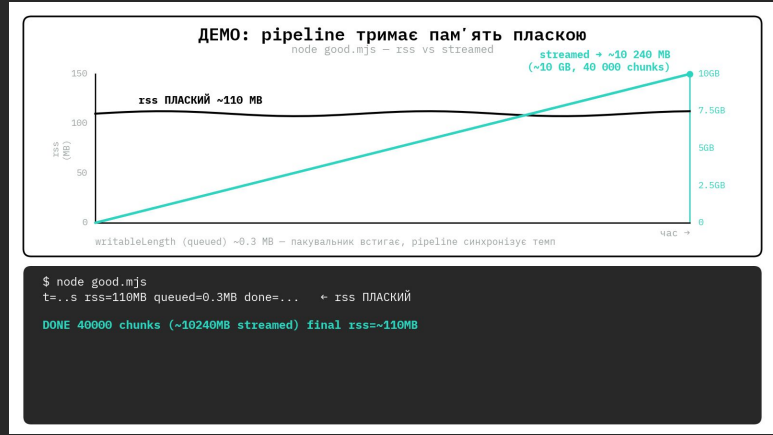
BAD — АКУМУЛЮЄМО ОБ’ЄКТИ БЕЗ ПЕРЕВІРКИ WRITE(): NODE --MAX-OLD-SPACE-SIZE=256 BAD-OBJECTS.MJS → FATAL ERROR: JAVASCRIPT HEAP OUT OF MEMORY




```
$ node --max-old-space-size=256 bad.js
t=0.1s rows=0.10M heapUsed=34MB   rss=104MB
t=0.2s rows=0.20M heapUsed=64MB   rss=141MB
t=0.6s rows=0.70M heapUsed=250MB  rss=~300MB
t=0.8s rows=0.90M heapUsed=252MB  rss=322MB

<--- Last few GCs --->
FATAL ERROR: Reached heap limit Allocation failed
- JavaScript heap out of memory   (exit 134, ~1.23s)
```

GOOD — PIPELINE(READ → TRANSFORM → SLOWWRITE): NODE GOOD.MJS → RSS=110MB QUEUED=0.3MB, DONE 40000 CHUNKS (~10240MB) FINAL RSS=~110MB




```
$ node good.mjs
t=..s rss=110MB queued=0.3MB done=...   ← rss ПЛАСКИЙ

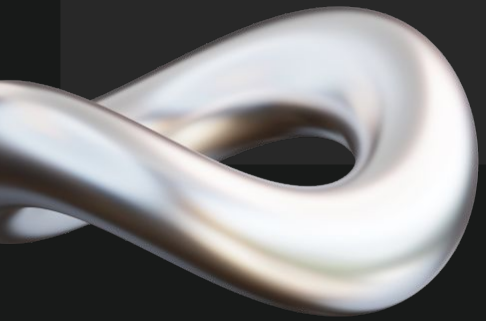
DONE 40000 chunks (~10240MB streamed) final rss=~110MB
```

Ліворуч: пам ’ять росте, поки процес не падає (~1 с). Праворуч: пам ’ять тримається ~110 MB незалежно від обсягу — ~10 GB крізь ~100 MB RAM

PIPELINE VS .PIPE() + УНІВЕРСАЛЬНИЙ BACKPRESSURE

.pipe() не пропагує помилку: якщо стадія падає, решта стрімів лишаються ВІДКРИТИМИ → leak дескрипторів (fd — file descriptor, сокети, пам ′ять)

Backpressure (зворотний тиск) — універсальний принцип потоків: те саме бачимо у TCP flow control, у Kafka consumer lag, у gRPC-стрімінгу



Правило проду: pipeline — завжди; .pipe() — ніколи

pipeline(a, b, c) при падінні закриває ВСІ стріми, повертає Promise (await + try/catch) і приймає { signal } (AbortSignal) для чистого скасування

WEB STREAMS VS NODE:STREAM VS RXJS

Три інструменти — три сценарії використання, не взаємозамінні

node:stream — рідні стріми Node з багатою екосистемою (zlib для стиску, fs для файлів); дефолтний вибір у звичайному Node-застосунку

Web Streams (ReadableStream / WritableStream) — крос-платформний веб-стандарт; єдиний варіант на edge-рантаймах (Cloudflare Workers, Vercel Edge, Deno), де node:stream відсутній; це і є fetch(url).body; Stable, без прапорців

RxJS — бібліотека для складних ПОДІЄВИХ потоків (autocomplete, WebSocket): оператори merge / debounce / retry — про час і події, а не про байти файлу

Не мішай web-стріми і node-стріми в одному модулі: копіювання на межі не безкоштовне, а типи й помилки поводяться по-різному

```
Міст між світами: Readable.fromWeb(res.body) — web → node (далі zlib/fs);
Readable.toWeb — node → web (для edge)
```

ПІДСУМКИ + МІСТ ДО L#3



- Async — керування ЧАСОМ: Promises → microtasks → AsyncLocalStorage (контекст запиту без прокидання параметрів) → AbortController
- Streams — керування ПАМ ′ ЯТТЮ: Readable / Writable / Transform → backpressure (зворотний тиск) → pipeline
- Наскрізна нитка обох половин — зворотний тиск: в async це AbortSignal, у streams — write() === false
- Тепер умієш: передбачити порядок виводу квізу (1, 5, 3, 4, 2); прокинути requestId через ALS без prop drilling; протягнути ~10 GB даних крізь ~100 MB RAM
- Міст до Лекції 3 (HTTP/TLS): ті самі байти поїдуть «по дроту» — розберемо TCP, HTTP/1.1 і TLS